Workflow Orchestration with Apache Airflow

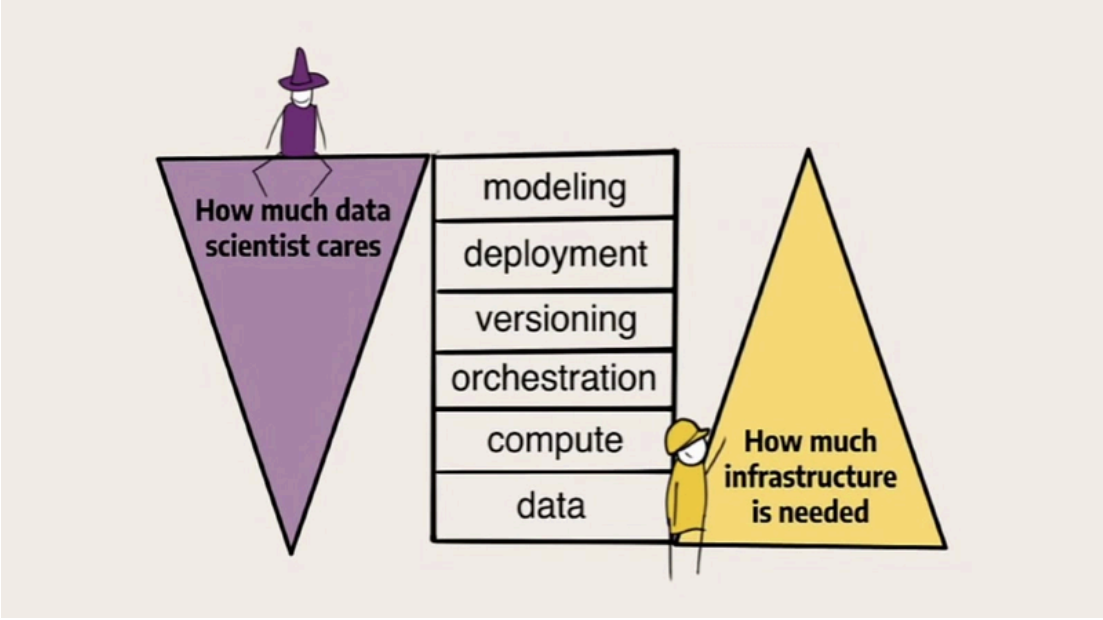
- **Course:** Engineering of Intelligent Models
- **Module:** M1. MLOps Foundations
- **Format:** Asynchronous Jupyter Notebook (self-study)
- **Focus Tool:** Apache Airflow

1. Learning Outcomes

After studying this notebook, you should be able to:

1. Explain why standard scheduling tools are insufficient for complex machine learning workflows.
2. Define the core architecture and components of Apache Airflow.
3. Understand the significance of Directed Acyclic Graphs (DAGs) in maintaining operational integrity.
4. Describe the interaction between Operators, Tasks, and XComs in a data-centric environment.
5. Conceptually integrate Apache Airflow into the Weather Forecast capstone project.

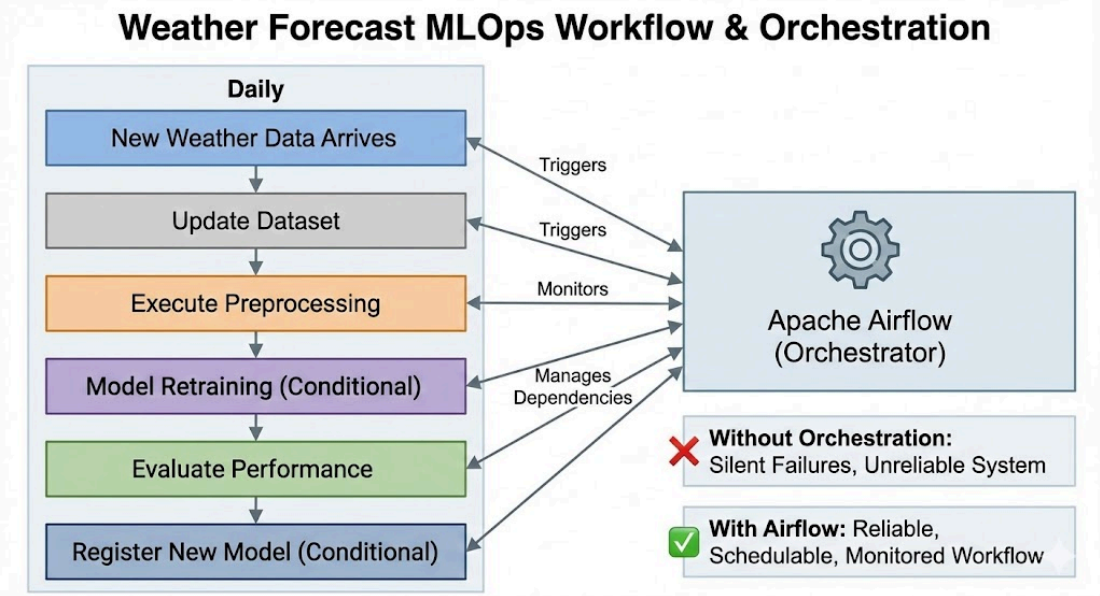
2. The Orchestration Problem: Why MLOps Needs More Than "Cron"



In traditional data environments, many engineers begin by managing workflows with simple scripts or time-based cron jobs. While cron is effective for basic, isolated tasks, it fails significantly when applied to the interdependent stages of a modern machine learning pipeline. Cron jobs operate without awareness of the steps that preceded them; for example, if a data ingestion script fails at 3:00 AM, a cron-based training script will trigger regardless, wasting computational resources on stale or non-existent data. This

lack of dependency management leads to unreliable execution and produces inconsistent results that are difficult to debug without centralized visibility.

Apache Airflow addresses these challenges by serving as a central orchestrator that coordinates multiple interdependent processes. Unlike static scheduling, Airflow provides a robust framework for defining explicit upstream and downstream relationships between tasks, ensuring they run only when their specific conditions are met. This programmatic approach allows engineers to move from error-prone manual intervention to a cohesive system characterized by automated retries, granular logging, and high-level observability through a web-based user interface.



Consider our Weather Forecast capstone project. Every day:

- New weather data arrives.
- The dataset must be updated.
- Preprocessing must be executed.
- The model may need retraining.
- Performance must be evaluated.
- If performance improves, a new model version may be registered.

If any of these steps fail silently, the system becomes unreliable. This introduces the need for **orchestration**. Orchestration is the discipline of defining, scheduling, monitoring, and managing complex workflows composed of interdependent tasks. Apache Airflow was designed to solve precisely this problem!

3. What is Apache Airflow?

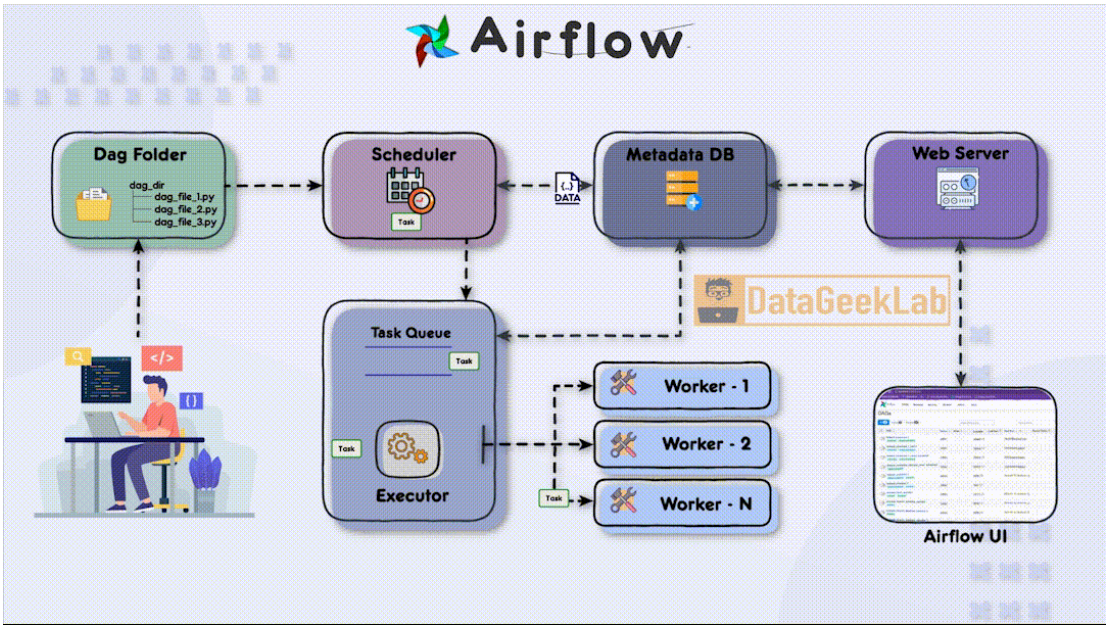
Apache Airflow is a leading open-source platform designed to programmatically author, schedule, and monitor workflows. Originally created at Airbnb to manage increasing complexity, it has since become the industry standard for automating and scaling data pipelines. Airflow's core philosophy is **"Configuration as Code,"** which means workflows are defined using Python scripts rather than static configuration files or drag-and-drop interfaces. This Python-native foundation makes the platform exceptionally extensible, allowing it to integrate seamlessly with various machine learning tools, cloud services, and CI/CD workflows.

3.1 The Architectural Components

To understand how Airflow handles the end-to-end lifecycle of a workflow, we must look at its distributed architecture. The platform consists of several critical components that work in harmony to ensure reliable execution:

- **The Scheduler**: This is the "brain" of the system. It monitors all DAGs and tasks, triggering task instances whose dependencies have been met and whose schedule has arrived.
- **The Executor**: While the scheduler decides *when* to run a task, the executor determines *how* to run it. It handles the queuing of tasks and distributes them to the available workers.
- **The Workers**: These are the actual execution units that perform the work defined in your tasks (e.g., executing a Python script to fetch weather data or running a DVC command).
- **The Metadata Database**: Typically powered by PostgreSQL or MySQL, this database stores the state of every task, DAG, and user, serving as the "source of truth" for the entire environment.
- **The Webserver**: This component provides a comprehensive User Interface (UI). Through the UI, engineers can inspect DAG structures, monitor real-time task progress, manually trigger runs, and debug failures via granular logs.

3.2 The Workflow Process



Retrieved from: Datageek Lab

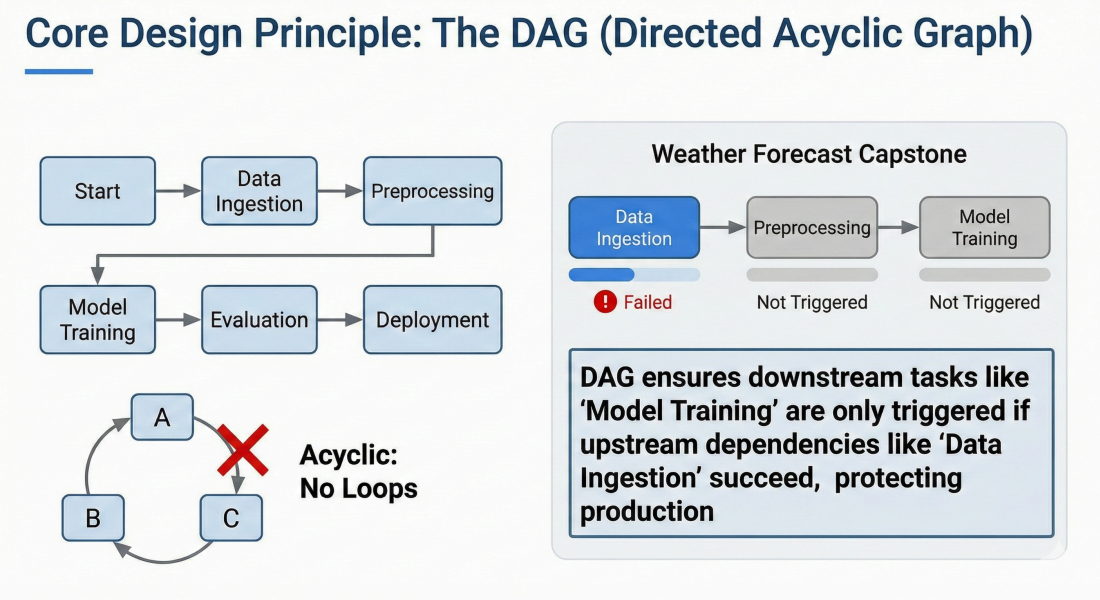
The interaction between these components follows a structured flow. First, the **Webserver** or **Scheduler** reads the DAG files from a dedicated folder. When a DAG is triggered, the **Scheduler** creates a task instance in the **Metadata Database** and notifies the **Executor**. The **Executor** then pushes the task to a queue where a **Worker** picks it up for execution. Once finished, the worker updates the task's final state (Success or Failed) back in the database so the scheduler can proceed with the next downstream steps in the graph.

This decoupled architecture is what allows Airflow to be highly scalable and resilient, as individual components can be distributed across multiple servers or containers without losing track of the global pipeline state.

4. The Core Design Principle: The DAG (Directed Acyclic Graph)

The fundamental concept in Airflow is the **DAG (Directed Acyclic Graph)**. A DAG is a collection of all the tasks you want to run, organised in a way that reflects their relationships and dependencies.

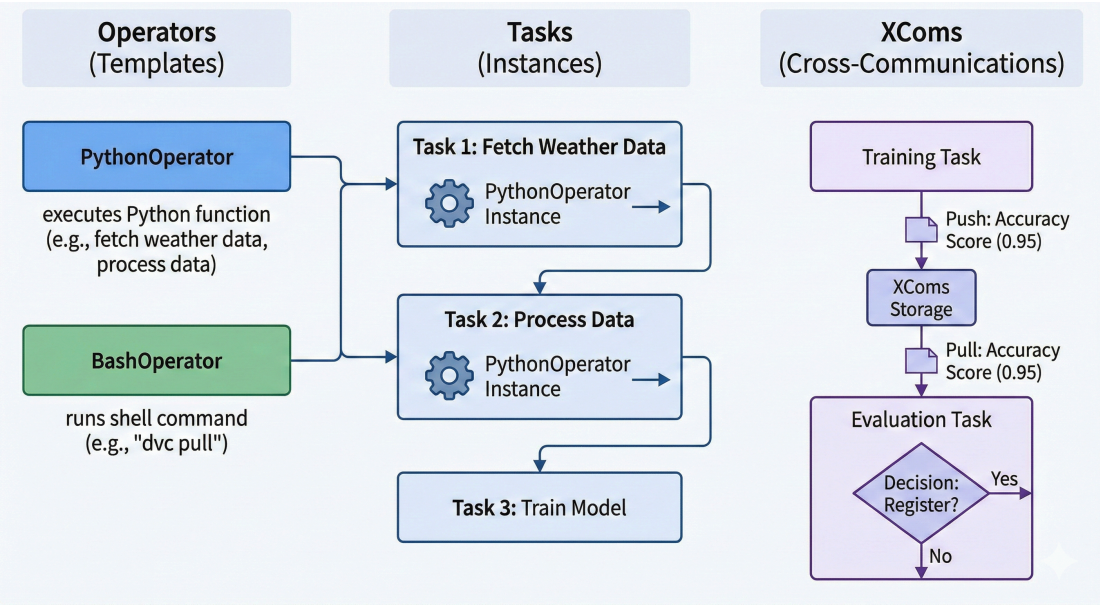
- **Directed:** The workflow has a clear beginning and end, with a specific direction of flow.
- **Acyclic:** There are no loops. Step A leads to Step B, but Step B cannot loop back to Step A.
- **Graph:** A visual representation of the logic.



By defining a DAG for our Weather Forecast capstone project, we ensure that if the "Data Ingestion" task fails, the "Model Training" task is never triggered, protecting our production environment from faulty updates.

5. Anatomy of a Workflow: Operators, Tasks, and XComs

To transform a high-level DAG into actual work, Airflow utilizes three primary concepts: **Operators**, **Tasks**, and **XComs**.



- **Operators:** These are the templates for a task. For example, a `PythonOperator` executes a Python function, while a `BashOperator` can run a DVC command like `dvc pull`. Operators are reusable and can be parameterised to perform different actions. For instance, you could have a `PythonOperator` that calls a function to fetch weather data from an API, and another `PythonOperator` that processes that data for model training.
- **Tasks:** Once an operator is instantiated with specific parameters, it becomes a task. A task is a single unit of work in the DAG. For instance, a task might be "Fetch

Weather Data," which uses a `PythonOperator` to run a function that calls an API and saves the data to a DVC-tracked directory.

- **XComs (Cross-Communications):** While tasks are generally isolated for reliability, XComs allow them to exchange small amounts of metadata, such as the location of a processed file or a model's accuracy score. This is crucial for dynamic workflows where the output of one task informs the execution of another. For example, after a model training task completes, it could push the model's performance metrics to an XCom, which a downstream evaluation task can then pull to decide whether to register the model in MLflow.

6. Airflow in the Weather Forecast Capstone

Our capstone requires periodic data updates to maintain forecast accuracy. Airflow will manage this lifecycle:

- **Schedule:** We can set the DAG to run hourly.
- **Ingestion Task:** Triggers a script to pull the latest meteorological data.
- **Processing Task:** Runs the feature engineering logic (only if ingestion succeeds).
- **Training Task:** Executes the **Prophet or LSTM** training script.
- **Deployment Task:** If evaluation metrics meet our "quality gate", the new model is pushed to the Model Registry.

This automation ensures that our weather service is always powered by the most recent data without manual intervention.

7. Recommended Further Reading

- Apache Airflow Official Documentation https://airflow.apache.org/docs/
- Airflow Concepts Guide https://airflow.apache.org/docs/apache-airflow/stable/concepts/index.html
- Huyen, C. (2022). Designing Machine Learning Systems: An Iterative Process for Production-Ready Applications. O'Reilly Media. (*published in Moodle*)